P0018r1 : Lambda Capture of *this by Value

Author: H. Carter Edwards
Contact: hcedwar@sandia.gov
Author: Christian Trott
Contact: crtrott@sandia.gov
Author: Hal Finkel
Contact: hfinkel@anl.gov
Author: Jim Reus
Contact: reus1@llnl.gov
Author: Robin Maffeo
Contact: robin.maffeo@amd.com
Author: Ben Sander
Contact: ben.sander@amd.com

Number: P0018
Version: 1
Date: 2015-10-23
URL: https://github.com/kokkos/ISO-CPP-Papers/blob/master/P0018.rst
WG21: Evolution Working Group (EWG)
WG21: Concurrency and Parallelism Study Group (SG1)

1 Issue

Lambda expressions declared within a non-static member function explicilty or implicitly captures the **this** pointer to access to member variables of **this**. Both capture-by-reference **[&]** and capture-by-value **[=]** *capture-defaults* implicitly capture the **this** pointer, therefore member variables are always accessed by reference via **this**. Thus the capture-default has no effect on the capture of **this**.

```
struct S {
  int x ;
  void f() {
    // The following lambda captures are currently identical
    auto a = [&]() { x = 42 ; } // OK: transformed to (*this).x
    auto b = [=]() { x = 43 ; } // OK: transformed to (*this).x
    a();
    assert( x == 42 );
    b();
    assert( x == 43 );
  }
};
```

2 Motivations for lambda capture of *this by Value

Truly capturing the ***this** object by value allows an implicitly declared closure to be copied before invoking the closure's function.

2.1 Asynchronous dispatch of lambda

Asynchronous dispatch of closures is a cornerstone of parallelism and concurrency.

When a lambda is asynchronously dispatched from within a non-static member function, via **std::async** or other concurrency / parallelism dispatch mechanism, the ***this** object *cannot* be captured by value. Thus when the **future** (or other handle) to the dispatched lambda outlives the originating class the lambda's captured **this** pointer is invalid.

```
class Work {
private:
  int value ;
public:
  Work() : value(42) {}
  std::future<int> spawn()
```

```
    { return std::async( [=]()->int{ return value ; }); }
};

std::future<int> foo()
{
  Work tmp ;
  return tmp.spawn();
  // The closure associated with the returned future
  // has an implicit this pointer that is invalid.
}

int main()
{
  std::future<int> f = foo();
  f.wait();
  // The following fails due to the
  // originating class having been destroyed
  assert( 42 == f.get() );
  return 0 ;
}
```

2.2 Dispatching asynchronous closures to data

Current and future hardware architectures specifically targeting parallelism and concurrency have heterogeneous memory systems. For example, NUMA regions, attached accelerator memory, and processing-in-memory (PIM) stacks. In these architectures it will often result in significantly improved performance if the closure is copied to the data upon which it operates, as opposed to moving the data to and from the closure.

For example, parallel execution of a closure on large data spanning NUMA regions will be more performant if a copy of that closure residing in the same NUMA region acts upon that data. If a true (self-contained) capture-by-value lambda closure were given to a parallel dispatch, such as in the parallelism technical specification, then the library could create copies of that closure within each NUMA region to improve data locality for the parallel computation. For another example, a closure dispatched to an attached accelerator with separate memory must be copied to the accelerator's memory before execution can occur. Thus current and future architectures *require* the capability to copy closures to data.

2.3 Onerous and error-prone work-around

A potential work-around for this deficiency is to explicitly capture a copy the originating class.

```
class Work {
private:
  int value ;
public:
  Work() : value(42) {}
  std::future<int> spawn()
    {
      return std::async( [=,tmp=*this]()->int{ return tmp.value ; });
    }
};
```

This work-around has two liabilities. First, the **this** pointer is also captured which provides a significant opportunity to erroneously reference a **this->** *member* instead of a **tmp.** *member* as there are two distinct objects in the closure that reference two distinct *member* of the same name. Second, it is onerous and counter-productive to the introduction of asynchronously dispatched lambda expressions within existing code. Consider the case of replacing a **for** loop within a non-static member function with a *parallel for each* construct as in the parallelism technical specification.

```
class Work {
public:
  void do_something() const {
    // for ( int i = 0 ; i < N ; ++i )
    foreach( Parallel , 0 , N , [=,tmp=*this]( int i )
    {
      // A modestly long loop body where
      // every reference to a member must be modified
      // for qualification with 'tmp.'
      // Any mistaken omissions will silently fail
      // as references via 'this->'.
    }
    );
  }
};
```

In this example every reference to a member in the pre-existing code must be modified to add the **tmp.** qualification. This onerous process must be repeated throughout an existing code base. A true lambda capture of ***this** would eliminate such an onerous and silent-error-prone process of injecting parallelism and concurrency into an large, existing code base.

2.4 Safety and productivity in parallelism and concurrency

As currently specified integration of lambda and concurrency capabilities is perilous, as demonstrated by the previous **Work** example. A lambda generated within a non-static member function *cannot* be a true (self-contained) closure and therefore cannot reliably be used with an asynchronous dispatch.

Lambda capability is a significant boon to productivity, especially when parallel or concurrent closures can be defined with lambdas as opposed to manually generated functors. If the capability to capture ***this** by value is not enabled then the productivity benefits of lambdas cannot be fully realized in the parallelism and concurrency domain.

3 Semantics of Lamda Capture of *this by value

Lambda captures of ***this** by value within a non-static member function is as if:

- the implicitly generated closure object includes a member instance of the type of ***this**,
- the enclosing function's ***this** is copy constructed into the corresponding member of the closure,
- name lookup (5.1.2p7) of **this**, ***this**, and non-static members of ***this** translate to the closure's copy of ***this**, and
- if the lambda is not mutable then the closure's copy of ***this** is **const**.

Requires: The type of ***this** to be copy constructable.

Requires: Lambda capture of ***this** by value cannot occur within a copy constructor or function invoked by a copy constructor. Such a circumstance would result in an infinite recursion of the copy constructor. Note that his requirement is also applicable to the onerous work-around of **[tmp=*this]**.

4 Impractical Preferred Solution: Correct Lambda Capture-by-value [=]

The semantically consistent solution is for the *capture-default* **[=]** to capture ***this** by value for lambda expressions within a non-static member function. The *capture-default* **[&]** within a non-static member function conforms to the current capture specification for **this**.

```
struct S {
  int x ;
  void f() {
    auto a = [&]() { x = 42 ; } // OK: transformed to (*this).x
    auto b = [=]() mutable { x = 42 ; } // Modifying copy of x

    auto c = [=]() { x = 42 ; } // Error: captured copy of '*this'
                                // and lambda function is 'const'
  }
};
```

This solution corrects lambda capture semantics; however, it is likely to break existing code conforming to the C++11 standard which depend on the copying of the **this** pointer rather than copying of the object to which the **this** pointer refers. As such we currently consider this solution to be impractical and propose the following pragmatic solution.

5 Pragmatic Solution: Add Lambda Capture-*this-by-value [=,*this]

Given that the semantically consistent preferred solution would break current standard behavior, a new capture mechanism is necessary to provide semantically consistent capture-by-value semantics for lambda expressions within non-status member functions.

5.1 Draft wording

Feature test macro: **__cpp_lambda_capture_this_object_by_value**

5.1.2p1: Extend the *simple-capture* to include ***this**.

simple-capture:
identifier
& *identifier*
this
***this**

5.1.2p8: Revise for capture of ***this** by value.

If a *lambda-capture* includes a *capture-default* that is = then the *lambda-capture* shall not contain **this**, the *lambda-capture* may contain ***this** to capture ***this** by value, and each remaining identifier the *lambda-capture* contains shall be preceeded by **&**. An identifier, **this**, or ***this** shall not appear more than once in a *lambda-capture*. Both **this** and ***this** shall not appear in the same *lambda-capture*. When ***this** object is captured by value the **this** pointer shall not be captured.

5.1.2 new paragraph

Explicit *lambda-capture* of ***this** requires that the type of ***this** is copy constructable. A *lambda-capture* of ***this** within the context of the copy constructor for the type of ***this** implicitly requires infinite recursion of that copy constructor and is thus ill-formed.

5.1.2p10: Modify for capture of ***this** by value.

An entity (i.e., a variable, **this**, or ***this**) is said to be ...

5.1.2p12: Modify to add ***this**

If a *lambda-expression* odr-uses (3.2) **this**, ***this**, or a variable with ...

5.1.12p15: insert

The ***this** entity is *captured by copy* if it is explicitly captured.

5.1.12p17: insert

If ***this** is captured, each odr-use of ***this** is transformed into an access to the corresponding unnamed data member of the closure type which is of the type of ***this**. [Note: Thus an odr-use of ***this** in a nested *lambda-expression* is likewise transformed. --end note]

5.2 Example for nested lambda expressions

A new capture mechanism introduces a new capture interaction. For non-***this** captures the interactions remain unchanged. When ***this** is captured by value via **[*this]** nested captures of **this** refer to the enclosing copy of ***this**.

```
void Work::foo()
{
  auto x = [=,*this]() {
    // this, *this, and member variables of Work refer to
    // the copy *this contained in closure 'x'
    // does not refer to the original enclosing 'this'
    auto y = [&]() {
      // this, *this, and member variables of Work refer to
      // the copy *this contained in closure 'x'
      // does not refer to the original enclosing 'this'
    };
    auto z = [=]() {
      // this, *this, and member variables of Work refer to
      // the copy *this contained in closure 'x'
      // does not refer to the original enclosing 'this'
    };
    auto zz = [=,*this]() {
      // this, *this, and member variables of Work refer to
      // a new copy *this contained in closure 'z'
      // that is copied from the copy of *this contained
      // in the closure 'x'
    };
  };
}
```

5.3 Updated example

With proper lambda capture-by-value the earlier examples can have the correct behavior by generating a complete closure.

```
class Work {
private:
  int value ;
public:
  Work() : value(42) {}
```

```
  // Capture-by-value is correct and the asynchronously
  // dispatched closure may outlive the originating class,
  // and may be freely copied without losing correctness.
  std::future<int> do_something() const {
    // Trivial change to replace 'for' with asynchronously
    // dispatched parallel foreach.
    // for ( int i = 0 ; i < N ; ++i )
    future<int> todo =
      foreach( Parallel , 0 , N , [=,*this]( int i )
      {
        // A non-trivial loop body where
        // every reference to a member is
        // safely accessed from the
        // captured-by-value *this
      });
    return todo ;
  }
};
```

6 Follow-up: capture-default [=*]

Assuming that correcting the *capture-default* behavior of **[=]** is impractical we seek a *capture-default* expression to simplify the true capture-by-value expression **[=,*this]**. A possible *capture-default* expression is to let **[=*]** be equivalent to **[=]** outside of a non-static member function and equivalent to **[=,*this]** within a non-static member function.